Question 52. □

Show the initial value problem

$$y' = y - t\cos ty, \quad 0 \le t \le 2, \quad y(0) = 2$$

has a unique solution and is also well-posed.

§4.2 One-Step Methods

Consider an IVP of the form

$$y'(t) = f(t, y(t)), \qquad \text{with } y(t_0) = y_0$$

Instead of trying to compute the value of $y(t)$ for all $t \in [t_0, \mathrm{T}]$, we will start by we choosing a set of evenly spaced points in time

$$t_0 < t_1 < t_2 < \ldots < t_{\mathrm{N}} = \mathrm{T}, \qquad t_i = t_0 + ih, \quad i = 0, 1, \ldots, \mathrm{N}, \quad h := \frac{\mathrm{T} - t_0}{\mathrm{N}}.$$

Our goal is to find a sequence of real numbers $\{y_0, y_1, \ldots, y_{\mathrm{N}}\}$ such that $y_i \approx y(t_i)$.

So we are essentially interested in finding algorithms that starts from $y_i \approx y(t_i)$ and finds

$$y(t_{i+1}) = y(t_i + h) = y(t_i) + \int_{t_i}^{t_i+h} f(t, y)\,\mathrm{d}t$$

Definition 4.2.29

A numerical method is called a one-step method if it can be written in the form:

$$y_{i+1} = y_i + h\Phi(t_i, y_i, y_{i+1}, h)$$

where $\Phi : [t_0, \mathrm{T}] \times \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is called the increment function.

The method is called **explicit** if Φ does not depend on y_{i+1}.

The term *one-step* indicates that computing y_{i+1} requires only information from the immediately preceding step (possibly through solving an equation involving y_{i+1}).

4.2.1 Taylor Series Methods

Since we know the function $y(t)$ at one input t_i and we know its derivative $y'(t)$ everywhere, we could consider its Taylor series at the known point t_i.

First, we must quantify the error introduced by our approximation.

Definition 4.2.30: Local Truncation Error

The **local truncation error** (LTE), denoted $\tau_{i+1}(h)$, is the error introduced in a single step of the method, normalized by the step size h, assuming the previous value was exact:

$$\tau_{i+1}(h) = \frac{y(t_{i+1}) - y(t_i)}{h} - \Phi(t_i, y(t_i), h)$$

Now, expand $y(t)$ in a Taylor series about $t = t_i$:

$$y(t) = y(t_i) + (t - t_i)y'(t_i) + \frac{(t - t_i)^2}{2}y''(c)$$

But the differential equation tells us that $y'(t_i) = f(t_i, y_i)$ so that:

$$y(t) = y(t_i) + (t - t_i)f(t_i, y_i) + \frac{(t - t_i)^2}{2}y''(c)$$

Therefore, we can compute $y(t_i + h) = y(t_{i+1})$ from:

$$y(t_{i+1}) = y(t_i) + \underbrace{(t_{i+1} - t_i)}_{=h} f(t_i, y_i) = y(t_i) + hf(t_i, y_i)$$

This is **Euler's method.** The general formula is:

$$y_{i+1} = y_i + f(t_i, y_i)h$$

By the definition of LTE above, if we assume $y(t)$ is twice continuously differentiable, the LTE for Euler's method is determined by the remainder term:

$$\tau_{i+1}(h) = \frac{h}{2}y''(\xi_i)$$

Since the error is proportional to h, Euler's method is **first-order accurate**, denoted $O(h)$.

Question 53.

Write code to implement Euler's method for initial value problems. Your function should accept as input a function $f(t, y)$, an initial condition, a start time, an end time, and the value of $h = \Delta t$. The output should be vectors for t and y that you can easily plot to show the numerical solution.

Note that the overall error for Euler's method is $\mathcal{O}(h)$, however, errors build up at each step, growing exponentially. As such, Eulers' method is highly inaccurate and rarely used in practice.

Question 54.

Use Euler's method with $h = 0.25, 0.125, 0.0625$ to estimate $y(1)$ for $y' = 2t + y, y(0) = 3$ which has the exact solution $y = 5e^t - 2t - 2$. Check that the numerical error varies approximately linearly with h.

Higher-order Taylor series methods

In theory, Taylor series methods with $\mathcal{O}(h^m)$ global error can be obtained for any power of m by truncating the Taylor series farther and farther out. For instance, a second-order Taylor series method would use an update formula like

$$y(t_{i+1}) = y(t_i) + hf(t_i, y_i) + \frac{y''(t_i)}{2}h^2$$

to compute the next approximate y-value. However, in order to evaluate the term $y''(t_i)$, we need to compute the second-derivative y'' symbolically. Higher-order Taylor series methods than this would need symbolic computations of even higher-order derivatives which become very difficult for complicated examples. Because of this, difference-quotient approximations are typically used in place of any second-order (and higher) derivatives in the Taylor series.

$$y''(t_i) = \frac{y'(t_{i+1}) - y'(t_i)}{h} + \mathcal{O}(h) = \frac{f(t_{i+1}, y_{i+1}) - f(t_i, y_i)}{h} + \mathcal{O}(h)$$

This results in $\mathcal{O}(h^3)$ local error and $\mathcal{O}(h^2)$ global error.

Question 55.

Consider the problem of evaluating of the integral

$$y(t) = \int_0^t \frac{1}{\sqrt{1+x^3}}\,\mathrm{d}x$$

(a) *Convert this problem into an IVP of the form*

$$y'(t) = f(t, y(t)), \quad y(0) = 0.$$

(b) *Solve the IVP by approximating $y(2)$ with 10 subdivisions using the second-order Taylor series method described above.*

4.2.2 Implicit One-Step Methods

Next let's take a look at some other ways to improve Euler's method. One way to think of Euler's method is that when you are approximating the integral $\int_{t_i}^{t_i+h} f(t,y)\,\mathrm{d}t$, we are using left-endpoint Riemann sum with a single rectangle. What happens if we approximate the quadrature instead with right-endpoint Riemann sum instead?

We get the equation

$$\boxed{y_{i+1} = y_i + hf(t_{i+1}, y_{i+1})}$$

called the **backward Euler method**. Because y_{i+1} depends on the value $f(t_{i+1}, y_{i+1})$, this scheme is called an *implicit method*; to compute y_{i+1}, one needs to solve a (generally nonlinear) system of equations (using Newton's method etc.), which is rather more involved than the simple update required for the forward Euler method.

Similarly, if we instead use the Trapezoid quadrature rule, we will get

$$\boxed{y_{i+1} = y_i + \frac{h}{2}\Big(f(t_i, y_i) + f(t_{i+1}, y_{i+1})\Big)}$$

Like the backward Euler method, the trapezoid rule is implicit. To obtain a similar explicit method, replace y_{i+1} by its approximation from the explicit Euler method:

$$\widetilde{y}_{i+1} = y_i + hf(t_i, y_i)$$

which results in

$$\boxed{y_{i+1} = y_i + \frac{h}{2}\Big(f(t_i, y_i) + f(t_{i+1}, \widetilde{y}_{i+1})\Big) = y_i + \frac{h}{2}\Big(f(t_i, y_i) + f\Big(t_i + h, y_i + hf(t_i, y_i)\Big)\Big)}$$

This is called **Heun's method** or the **improved Euler's method**.

Predictor-Corrector Methods: The above trick of turning the implicit method to an explicit method gives a class of algorithms in numerical analysis known as predictor–corrector methods. All such algorithms proceed in two steps:

- The initial, "prediction" step, starts from a function fitted to the function-values and derivative-values at a preceding set of points to extrapolate ("anticipate") this function's value at a subsequent, new point.
- The next, "corrector" step refines the initial approximation by using the *predicted* value of the function and *another method* to interpolate that unknown function's value at the **same** subsequent point.

We will see some more examples in the next section.

4.2.3 Runge-Kutta Methods

Let's start by restating Heun's method in a form that is more easier to read. Define

$$k_1 = f(t_i, y_i)$$
$$k_2 = f(t_i + h, y_i + hk_1)$$

So k_1 is the slope of $y(t)$ at the beginning of the interval $[t_i, t_{i+1}]$, and k_2 corresponds to the slope of the solution one would get by taking one Euler step with stepsize h starting from (t_i, y_i).

The improved Euler's method, which is also known as the classical Runge-Kutta method of order 2 (or RK2) now says

$$y_{i+1} = y_i + \frac{h}{2}\Big(k_1 + k_2\Big)$$

Additional function evaluations can deliver increasingly accurate explicit one-step methods, an important family of which are known as Runge–Kutta methods.

General Explicit Runge-Kutta methods

General explicit Runge-Kutta methods are of the form

$$y_{i+1} = y_i + h\sum_{j=1}^{n} b_j k_j$$

with

$$\begin{aligned}
k_1 &= f\left(t_i + 0h, y_i\right)\\
k_2 &= f\left(t_i + c_2 h, y_i + h a_{21} k_1\right)\\
k_3 &= f\left(t_i + c_3 h, y_i + h a_{31} k_1 + h a_{32} k_2\right)\\
&\vdots\\
k_n &= f\left(t_i + c_n h, y_i + h\sum_{j=1}^{n-1} a_{n,j} k_j\right)
\end{aligned}$$

Determination of the coefficients is rather complicated and requires multivariate Taylor series computations. We will look at some specific cases below.

To specify a particular method, one needs to provide the integer n (the number of stages), and the coefficients a_{ij}, b_l and c_l. The matrix $\mathrm{A} = [a_{ij}]$ is called the Runge–Kutta matrix, while the b_i and c_i are known as the weights and the nodes. These data are usually arranged in a mnemonic device, known as a Butcher tableau:

$$\begin{array}{c|cccccc}
0 & & & & & & \\
c_2 & a_{21} & & & & & \\
c_3 & a_{31} & a_{32} & & & & \\
\vdots & \vdots & \vdots & \ddots & & & \\
c_n & a_{n1} & a_{n2} & \cdots & a_{n,n-1} & & \\
\hline
 & b_1 & b_2 & \cdots & b_{n-1} & b_n
\end{array} \quad = \quad \begin{array}{c|c} \vec{c} & \mathrm{A} \\ \hline & \vec{b}^{\,\mathrm{T}} \end{array}$$

where $c_1 = 0$ and $a_{ij} = 0$ for $i \leq j$.

Question 56.

Check that the improved Euler's method above corresponds to the tableau

$$\begin{array}{c|cc}
0 & 0 & 0 \\
1 & 1 & 0 \\
\hline
 & \frac{1}{2} & \frac{1}{2}
\end{array}.$$

In general, the Butcher tableau for a RK2 method looks like

$$\begin{array}{c|cc}
0 & 0 & 0 \\
\alpha & \alpha & 0 \\
\hline
 & \left(1 - \frac{1}{2\alpha}\right) & \frac{1}{2\alpha}
\end{array}.$$

The case of $\alpha = \dfrac{1}{2}$ is called the **Midpoint method**. The case of $\alpha = \dfrac{2}{3}$ is called the **Ralston method**. All

of these methods require 2 stages per time step and have a local truncation error of $\mathcal{O}(h^3)$ which leads to a global error of $\mathcal{O}(h^2)$.

Classical RK4

The classical fourth order Runge-Kutta method is given by the Butcher tableau

$$\begin{array}{c|cccc} 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6}. \end{array}$$

It requires four evaluations of the function f per time step, and has a local truncation error of $\mathcal{O}(h^5)$.

Question 57.

Write down the classical RK4 in an algorithm form and write a code to implement it. Test the problem on several differential equations where you know the solution.

Here's some pseudocode to help you get started:

CODE:

```
def rk4(f,x0,t0,tmax,dt):
    t = # build the time array
    x = # initialize an empty array for the x values
    x[0] = # build the initial condition
    # On the next line: be careful about how far you're looping
    for i in range( ??? ):
    # The interesting part of the code goes here.
    # Start from t[i] and x[i]. Then Define k1, k2, k3, k4
    # and use them to find x[i+1]
    return t, x

f = lambda t, x: # the definition of the function goes here

x0 = # initial condition
t0 = 0
tmax = # your choice
dt = # pick something reasonable
t, x = rk4( ??? , ??? , ??? , ??? , ??? )
# plot t vs. x
```